# RecoverML: Exact Artifact Time Travel for ML Pipelines under Declared Capability Loss

Abdul Hannaan
Qatar University
Doha, Qatar
aa2007325@qu.edu.qa

## Abstract

Machine learning experiments leave related versions of data, models and predictions, and recovering an old result may require a processing step that no longer works. RecoverML selects one shared set of files under a declared recovery contract and checks restored bytes against captured hashes. Its exact planners certify the minimum for the stated physical retention model. In a 36-history shared preprocessing study, RecoverML stores less than strong local search in nine cases and ties in 27, while a manual best frontier ties every minimum. We later added exact delta encoding and joint compression. Across 446 related storage conditions, the final five-candidate choice improves on the previous flat optimum in 244 of 444 measurable cases and ties in 200, with a largest reduction of 89.84%. Against local search with the same offered formats, it wins 20, ties 424 and loses none; two Letter conditions remain unmeasured. The median saving against that local control is zero, and an equal-format exact reference still matches the certified minimum. A separate native DVC recovery census gives both methods the same optimal retained payloads and finds a median paired elapsed-time advantage for RecoverML in all 864 tested configurations. The advantage concerns the measured DVC integration because a direct executor is near parity. These results support a checked historical recovery system with specific storage and orchestration gains rather than a new general optimization or compression algorithm.

## Keywords

historical machine learning recovery, data provenance, artifact retention, materialization, capability loss, reproducibility

## 1 Introduction

Machine learning research rarely ends with one execution because researchers change the data and features and model settings as they investigate a question. Each execution can leave processed datasets and fitted preprocessing state and trained models and predictions, and several versions may share much of the same work. Systems such as HELIX and HYPPO already use these shared computations to reduce repeated execution, while collaborative workload optimizers organize artifacts across experiments so that later work can reuse them [1, 7, 18].

Earlier versions can also matter after a new model has been chosen because a researcher may need to inspect a processing error or recover the output behind an earlier result. The EyeAI case described by Li et al. provides a concrete example: the project recorded more than 70 datasets and 130 executions and 142 trained models, and its provenance helped trace poor diagnosis performance to an upstream optic nerve extraction step [10]. The need to inspect that history motivates preservation, although it does not establish how much storage a retention optimizer would save.

Consider an image workflow in which raw images pass through a feature extractor and then a training step produces a model and predictions. If the extractor changes, an earlier feature file may still be needed to explain the old predictions. Keeping only the images and the current extractor cannot reproduce those earlier features when the old extractor is unavailable. Keeping the earlier features can preserve downstream recovery when the recorded training and prediction steps still work, but a contract that also allows those steps to fail may require retaining the models or final outputs themselves. The right files to keep therefore depend on the particular historical outputs being protected and the processing capabilities assumed to survive.

Pipeline tools already support useful parts of this process. DVC records dependencies and outputs and can freeze selected stages so that reproduction does not execute them [3], and Kishu restores earlier notebook states through incremental checkpoints [9]. Model persistence also has an environment boundary because loading an old model may require compatible software versions [15]. RecoverML addresses a specific decision within this setting: given a captured history and a declared set of possible processing losses, which files should remain stored so that every protected output can still be recovered exactly?

We express this decision as a recovery contract. The contract identifies protected targets and mandatory source files and groups of processing capabilities that may become unavailable together. For every allowed loss, each target must either be retained or have a valid recovery path through retained files and surviving producers. Recorded hashes identify the expected file bytes, and physical payloads with the same identity are charged once together with the required metadata. A set that works for one loss but fails for another allowed loss does not satisfy the contract. The promise assumes complete recorded dependencies and intact retained files and the supported replay environment.

The planner uses exact formulations for the general recorded model and applies a minimum cut specialization when the history satisfies its eligibility conditions. The specialization uses established flow techniques rather than a new graph optimization algorithm [12], and its certificate contains a feasible integer flow and a retained cut with equal cost. A separate checker verifies this evidence and the recovery executor checks the resulting files against their expected hashes. When every requested target is already retained, a direct file export path verifies and returns those bytes without loading a model or starting numerical libraries. File export preserves byte identity and does not imply that an incompatible environment can execute the serialized model.

Our evaluation separates storage selection from recovery execution and keeps strong baselines and manual controls visible [4]. In an exploratory grid using two datasets, 36 captured histories are assessed under both separate and shared preprocessing loss, giving 72 paired history and contract cases. Under shared loss the exact planner improves storage over strong local search in nine cases and ties in 27, with a maximum reduction of 48.74%. All 18 cases with an explicitly stored shared concatenation file tie local search, which shows that the benefit depends on how intermediate work is captured. The manually chosen best frontier matches every exact minimum.

A separate native DVC experiment gives both systems the same optimal retained payloads and uses one batched downstream DVC stage. With three paired repetitions per history, RecoverML has a lower median elapsed time in all 36 histories and the median DVC to RecoverML elapsed time ratio is 2.19 for the 18 histories needing recomputation and 9.91 for the 18 histories whose targets are all retained. The timing includes subprocess startup and final hash verification and excludes setup and retention planning, so it measures these command line integrations rather than an inherent difference between all deployments of the two systems. Earlier mixed outcomes remain in the artifact as separate development revisions rather than additional trials.

The later storage study changes how retained artifacts are represented. It offers full files, exact prefix and suffix deltas, and joint archive compression, then chooses the smallest complete store from five computed candidates. Across 446 related conditions this choice reduces bytes against the earlier flat optimum in 244 of 444 measurable comparisons, but it improves on local search with the same offered formats in only 20 and ties in 424. A broader native DVC census also measures recovery at equal optimal retained payloads over 864 configurations. We report these complete grids, including the two unmeasured Letter cases and the direct executor control, because the advantages depend on the comparator and objective.

The larger Letter experiment shows another boundary: selective retention saves 3.6647 decimal GB of uncompressed canonical bytes under preprocessing loss, but the saving is 290.61 MB with the measured gzip representation and zero under broader training and prediction loss. Strong robust heuristics match the storage saving and loading snapshots is faster in that descriptive recovery run. These controls help distinguish the value of retaining fewer bytes from the separate questions of optimization difficulty and recovery speed.

*Novelty and contributions.* The contribution of RecoverML is the integration of a declared historical preservation contract with checked retention planning and executable recovery, together with evidence showing how correlated processing loss and capture boundaries change the useful choices. We build on existing provenance and materialization and optimization methods, and the resulting system and measurements make three contributions:

(1) **An explicit contract for exact historical recovery.** We connect protected artifact identities and mandatory roots and correlated capability losses to a storage objective that counts physical payloads and required metadata. The contract makes the recovery obligation and its assumptions checkable.
(2) **A checked retention and recovery system.** We combine general exact formulations with component decomposition and a guarded minimum cut specialization, and provide certificate checks and recovery verified against recorded hashes. A direct export path handles fully retained targets without unnecessary model loading.
(3) **An empirical account of the useful choices and their limits.** Public data and retinal image workflows measure exact recovery, storage, planning and restoration. Paired local and manual controls expose the effects of failure grouping and capture granularity. Later full-catalogue storage and native DVC recovery studies test exact encoding choices and recovery latency at equal retained payloads, while reporting ties and weaker direct-executor outcomes.

## 2 Related Work

### 2.1 Materialization and reuse across ML experiments

HELIX separates the decision to retain an intermediate result from the decision to load or recompute it in a later iteration, and it uses a flow formulation for reuse planning and heuristics for materialization [18]. Its treatment of iteration and intermediate files is a direct precedent for this work. RecoverML instead fixes the historical targets that must remain recoverable and asks for the smallest retained store that satisfies declared processing losses, so a useful reuse plan alone is not sufficient to establish its preservation obligation.

Derakhshan et al. organize collaborative workloads in an Experiment Graph and select artifacts using information such as size and computation cost and reuse frequency [1]. Their later production pipeline work groups shared computation in a DAG and chooses materializations under a storage budget to reduce later execution time [2]. These studies already establish shared history and storage aware execution planning. RecoverML uses related graph information but evaluates complete recoverability under the stated losses rather than future reuse likelihood or model utility.

HYPPO represents alternative ways to produce artifacts in a directed hypergraph and uses equivalences to find less expensive execution plans [7]. The later Hyppo demonstration adds interactive discovery and exploration of shared pipeline histories [8]. These systems are close to RecoverML in their use of history and alternative producers, and our distinction is the explicit preservation obligation and its checks. The experiments use pinned HYPPO materialization components and a HELIX rule adapter through a common executor, which supports a comparison of those policies under a declared budget but does not reproduce the complete native systems. We also compare robust greedy and local search policies because a heuristic designed for the same obligation is a stronger decision quality baseline than a policy designed for a different reuse objective.

## 2.2 Scientific provenance and historical state recovery

Li et al. link datasets and features and executions and assets in a versioned scientific workflow, and the EyeAI application shows how these records can support investigation of an earlier feature extraction problem [10]. That work motivates the need for historical access and provides a broader framework for collaborative provenance. RecoverML studies which captured files can be removed while preserving a specified set of historical outputs under declared producer loss. Our retinal experiment is a separate public image adaptation inspired by this use case and is not an evaluation on the private EyeAI clinical workflow.

Kishu provides time travel for computational notebooks by preserving dependencies between variables and restoring earlier states through incremental checkpoint and checkout operations [9]. Exact historical restoration is therefore already an explicit goal in prior systems. RecoverML protects selected immutable artifact files under a producer loss contract rather than restoring the complete interactive notebook state, and the two systems address different preservation boundaries.

DVC provides dependency tracking and stage reproduction, and frozen stages can preserve chosen intermediate outputs without rerunning their producers [3]. A manually chosen DVC frontier can therefore implement the same retained file choice as RecoverML, which our native controls confirm. RecoverML adds automatic selection and certificate checks around the declared contract, while the separate native latency experiment measures the cost of executing two authored integrations with the same paid payloads. It does not establish that DVC lacks historical recovery or that an optimal DVC frontier needs more storage.

## 2.3 Replay and checkpoint tradeoffs

CHEX uses execution lineage and ordered checkpoints to share computation while replaying multiple program versions [11]. Its cache and replay sequence decisions are closely related to the cost of returning to earlier work, although RecoverML selects persistent files to satisfy a recovery obligation after declared producers become unavailable. Checkmate uses mixed integer optimization to trade neural training computation for activation memory [6]. That result also shows why using an integer optimizer is not itself a new contribution, and its training memory objective differs from preserving historical artifact bytes.

ExoFlow separates execution from recovery and uses task annotations to support recovery choices with exactly-once workflow semantics [19]. Explicit recovery assumptions and the cost of checkpointing are therefore established systems concerns. RecoverML verifies immutable output identities within a supported replay environment and does not provide distributed exactly-once side effects or general crash consistency. Its storage benefit and its execution cost must be measured separately because retaining an earlier intermediate can save bytes while making later recovery more expensive.

## 2.4 Optimization foundations and the remaining distinction

Implicit hitting set methods already combine a candidate selection problem with an oracle and learned constraints, including applications to propositional abduction [13]. Minimum cost source location also studies the selection of paid sources under connectivity requirements [14], and vertex splitting is an established way to express vertex costs in a flow network [12]. These are relevant foundations for retention planning and constrain claims of algorithmic originality.

RecoverML applies these foundations to a recorded ML artifact contract and checks that the selected files preserve its protected targets under every allowed loss. Its supported DAG specialization avoids a general integer formulation and provides checkable flow and cut evidence, while the general route handles recorded structures outside that guard. The measured shared loss cases show that strong local search can miss a smaller feasible store, but the manual frontier ties and explicit concatenation controls show that some histories have simple optimal choices. The contribution is a checked preservation service and an account of when its choices matter, rather than a claim that every ML history requires difficult optimization or that the underlying graph methods are new.

# 3 Methodology

RecoverML first records the experiment history and then separates the decision about which bytes to retain from the execution of a later recovery request. Figure 1 shows this path. The planner works with recorded dependencies and a declared failure contract, and the executor receives only the selected store and its manifest. The original capture archive is kept for experimental checks but is not an available source during recovery.

## 3.1 Captured history and physical file identities

A logical node records an operation and its input node identifiers and parameters, together with its output hash and byte size and replay support. The environment record includes the numerical package versions and implementation hashes used by supported operators. Arrays use the recorded binary serialization and supported model classes use the project model codec, so the recovery claim concerns those captured bytes rather than a model with merely similar accuracy.

Several logical nodes can refer to the same physical payload, and the planner must not charge that payload once for each occurrence. Let $B$ be the captured physical blobs and let $w_b$ be the nonnegative size of blob $b$. The mapping $\beta(p)$ identifies the physical output of producer $p$, and $D(p)$ is the set of physical inputs it requires. Multiple recorded producers can therefore supply one blob, while an operation requiring several inputs needs all of them. Nonreplayable nodes keep their provenance but do not create executable recovery paths.

Mandatory roots $R \subseteq B$ must be retained and the protected targets are $T \subseteq B$. Targets can include models and probability arrays or earlier features needed for investigation. All methods in a comparison use the same captured manifest and root obligations. A method cannot read an omitted file from the capture archive, infer

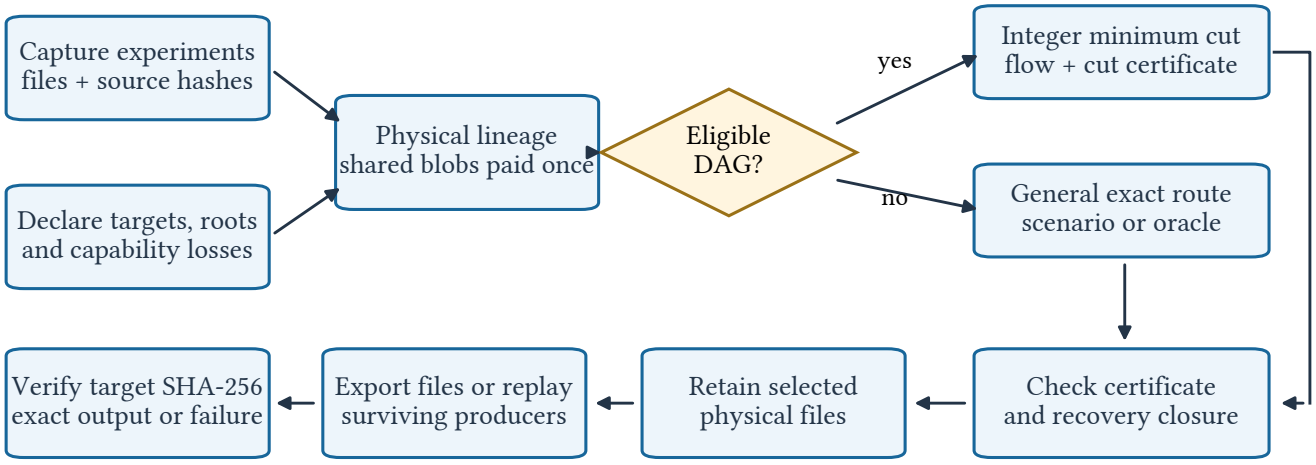



**Figure 1: RecoverML methodology. The contract and captured history define one retention problem, and a graph guard selects a supported exact route. A checked store is then used for direct file export or grounded replay under the actual declared loss. Hash verification is required before reporting an exact result. The minimum cut route applies only to the stated DAG subclass.**

missing bytes from their hash or silently replace a target with a newer model.

## 3.2 Recovery contract and byte objective

The capability map assigns a replayable producer to a declared group, and several producers can share a group when they rely on the same unavailable entry point. Ungrouped producers remain available. For group set $G$ and bound $k$, the allowed losses are

$$\mathcal{F}_k = \{F \subseteq G : |F| \le k\}. \tag{1}$$

The contract covers processing loss while retained files and mandatory roots remain intact. A changed environment or a corrupt retained file can cause a refusal rather than a recovery guarantee.

For retained set $S$, grounded recovery begins with $A_0 = S$ and repeatedly adds outputs of surviving producers whose inputs are already available:

$$A_{i+1} = A_i \cup \{\beta(p) : p \text{ survives } F,\ D(p) \subseteq A_i\}. \tag{2}$$

The process stops at the least fixed point $\mathrm{cl}(S, F)$. Beginning from retained values prevents an unsupported cycle from counting as a recovery path. The complete preservation problem is

$$\begin{aligned} \min_{S \subseteq B} \quad & m + \sum_{b \in S} w_b \\ \text{subject to} \quad & R \subseteq S, \\ & T \subseteq \mathrm{cl}(S, F) \quad \forall F \in \mathcal{F}_k, \end{aligned} \tag{3}$$

where $m$ is the fixed manifest and serialized contract size. Roots and metadata are included in reported retained bytes and paid once. A separately imposed byte budget can make the complete contract infeasible, and maximizing partial recovery below that minimum is a different objective.

## 3.3 General exact formulations

The scenario formulation shares binary retain variables across the allowed losses and adds availability and producer choices for each scenario. If a producer is used, every input must be available and the producer's output must have a later rank than its inputs. These ranks rule out ungrounded cyclic explanations. The formulation minimizes the payload term in Equation 3 and checks the selected store using grounded recovery after solving. Enumerating $\mathcal{F}_k$ can become expensive as the group count and loss bound grow, and the reference implementation refuses grids above its declared scenario limit.

The second formulation uses a storage master with one retain variable $x_b$ per physical blob and a separation oracle. The oracle looks for an allowed loss $F$ and a closed set $C$ that contains the selected files but omits at least one target. Closure means that every surviving producer with all inputs in $C$ also has its output in $C$. Such a witness proves that the selected store cannot recover every target for that loss, and every valid store must then satisfy

$$\sum_{b \in B \setminus C} x_b \ge 1. \tag{4}$$

The master adds this necessary constraint and chooses another minimum cost candidate. If the master optimum is proved and the oracle proves that no such witness exists, the store is robust and storage optimal within the recorded model. A timeout or an invalid incumbent is recorded as unknown rather than a certificate. This candidate and oracle design follows established implicit hitting set and abduction ideas [13].

The implementation checks binary values and the objective and roots and learned constraints before accepting an incumbent. The automatic dispatcher first attempts the DAG specialization below and otherwise chooses the scenario route when there are at most 64 allowed scenarios and the master and oracle route above that

threshold. The threshold is an engineering choice rather than a guarantee of the fastest solver, and solver status and postchecks remain necessary for either general route.

## 3.4 Guarded DAG specialization and its certificate

The specialized route requires a physical dependency DAG after mandatory roots are treated as available, and each optional blob must have at most one replayable producer. The guard rejects physical cycles and multiple replayable producers for the same output. Physical identity matters here because two logical producers with identical output bytes still violate producer uniqueness.

For this subclass, universal recovery when $k \geq 1$ is equivalent to recovery with all grouped producers disabled together:

$$\bigcap_{F \in \mathcal{F}_k} \mathrm{cl}(S, F) = \mathrm{cl}(S, G). \tag{5}$$

This is a reasoning equivalence for a fixed store and does not declare the simultaneous loss of all groups to be an allowed runtime failure. To see why it holds, consider the blobs in topological order. A retained blob is always available, an unretained blob whose sole producer can fail is not universally available, and a blob with an ungrouped sole producer is universally available exactly when all its inputs are. These are the same rules obtained by disabling all grouped producers. For $k = 0$, no producer is disabled.

After that reduction, an unretained blob with no surviving producer is a failure source. Failure can propagate along a dependency edge if the consuming operation needs the unavailable input, while retaining a blob stops propagation at that point. Protecting the targets therefore becomes a weighted vertex cut between failure sources and targets. We split each optional blob into input and output vertices joined by a capacity-$w_b$ edge, and dependency edges and terminal connections receive capacity

$$M = 1 + \sum_{b \in B \setminus R} w_b. \tag{6}$$

An optimum cannot cut an $M$ edge because retaining all optional blobs costs less. This is the standard vertex-splitting construction applied to the stated preservation model [12], and roots and fixed metadata are added to the selected payload cost.

The implementation obtains an integer maximum flow and a corresponding cut using NetworkX. Its checker validates every flow edge and capacity and conservation constraint, then removes the selected retained vertices and checks that no failure source reaches a protected target. The optional retained weight must equal the feasible flow value. Feasibility of the retained cut gives an upper bound and feasibility of the flow gives a lower bound, so equality certifies the optimum without rerunning the optimizer. A separate grounded-closure check verifies that the selected store actually protects the targets.

The equivalence does not extend to arbitrary AND/OR histories. If two producers in different groups can create one target from a retained root, either producer may survive any single group loss even though disabling both destroys recovery. An ungrounded physical cycle also breaks the failure source argument. These cases explain why the eligibility guard and general fallback are part of the method.

## 3.5 Components and recovery execution

An optional decomposition removes mandatory roots from the live dependency connectivity graph and solves components with targets separately. Their selected payloads are combined while the original global metadata and roots are paid once, and internal solver projections are not written as capture manifests. Shared failure labels do not invalidate this decomposition under the universal contract because each component must satisfy every allowed loss of its own relevant groups. A single optional target with only producers that depend only on roots also has a direct rule: it need not be retained if an ungrouped producer exists or the number of distinct producer groups exceeds $k$.

The executor builds a grounded witness from the actual retained store and surviving producer view and follows it in dependency order. Each request has its own memo and trace, and recovered or loaded payloads must match their recorded hashes. Missing files and source mismatches and corrupt retained bytes are reported explicitly. Stores are trusted local artifacts and the general restorer checks its supported runtime contract before loading values, and the execution never treats an unavailable producer as a usable alternative.

When all requested targets are already retained, the standard-library export route verifies their sizes and SHA-256 identities and materializes the output files without deserializing model objects. If a target is absent it returns control to ordinary replay, and if a retained target is corrupt it fails rather than bypassing the error. Exact file export is distinct from executing an old model in an incompatible environment. These are immutable-artifact recovery rules and do not establish distributed crash consistency or exactly-once external side effects.

# 4 Experimental Setup

We ask whether the declared loss contract changes retained storage and whether strong heuristics make the same choice, then measure planning cost and actual recovery cost separately. The studies use fixed public-data histories and all their recorded settings rather than a sample of independently collected production traces. Table 1 gives their scope and the artifact preserves the raw protocols and source snapshots [4].

## 4.1 Histories and declared processing changes

The new shared loss grid uses Wine and Breast Cancer from scikit-learn [16]. We split observations into 75% training and 25% test data using stratification and the three fixed seeds 20261016, 20261017 and 20261018. Features are divided into two disjoint column views and each view is standardized using only training observations. Logistic regression versions span a fixed regularization range from $10^{-3}$ to $10^{3}$, and every configured version is kept. In the fused boundary the downstream stages consume both views directly, while the other boundary records their concatenated matrix as an additional artifact.

Each captured history is tested with separate preprocessing groups and then with one shared group covering both standardization entry points, with $k = 1$. The same captured physical bytes are used across the paired contracts. The shared contract is an exploratory follow-up motivated by a disclosed favorable pilot,

**Table 1: Study design. A capture is a recorded history, and several contracts or policies can reuse that capture. These rows must not be added as independent application samples. The concurrency control measures the earlier prototype.**

| Study | Data and model family | Captures; versions | Main comparison or control |
|---|---|---|---|
| Shared-loss study | Wine and Breast Cancer; logistic regression | 36; 4/16/64 | Three seeds, two capture boundaries, two failure groupings, nine policies |
| Native recovery | Fresh captures of the same recipe grid | 36; 4/16/64 | RecoverML versus native batched DVC at identical paid payloads; three paired repetitions |
| Encoded storage | Six public datasets plus scientific, cloud and retinal transfers | 446 conditions | Previous flat and delta choices, flat joint archive and equal-format local search; two Letter cases unmeasured |
| Recovery census | Six public datasets and four deterministic recipes | 72 captures | Three loss contracts, two request sizes and two interfaces; 864 paired configurations at equal paid payloads |
| Scientific matrix | Iris, Digits and Diabetes; four recipes | 8; 3 each | Two seeds, two loss scopes, three bounds, five budgets, eight policies |
| Retinal histories | 20 STARE images; logistic regression and forest | 18; 2/8/16 | Two target modes, three availability contracts, five retention policies |
| Large payload | UCI Letter; trained random forests | 1; 16 | Preprocessing versus broad loss and canonical versus compressed representation |
| Earlier concurrency | Synthetic logistic regression histories | 2; 5 each | 2,000/10,000 rows; 1/2/4 threads; three repeats; 100 requests per trial |

and both small histories and explicit-concat controls remain in the full grid. No production outage frequency is estimated.

The scientific matrix has four recipe adaptations: Iris feature union, Digits PCA, Diabetes polynomial features and Digits forests. Two seeds and three versions give eight captures, and the matrix varies preprocessing versus broad loss with $k \in \{0, 1, 2\}$. Its five budgets are based on fixed root and metadata costs plus 0, 10, 25, 50 or 100% of optional payload bytes of target snapshots, so the budget construction does not depend on an optimum found later. Across eight methods this gives 1,920 method and budget rows.

The retinal study uses all 20 STARE fundus images and Hoover vessel annotations [5]. Images are resized to $64 \times 64$ RGB and split by image into 15 training and five test images, with 6,000 sampled training pixels. Three seeds, two model families and three history lengths give 18 captures. A legacy three-pixel feature shift is replaced by an aligned extractor and the two revisions must produce different feature bytes. Targets are either models and predictions or those files plus investigation features, and the contracts cover intact processing, loss of the legacy extractor or loss of training and prediction. Model settings also change across versions, so the experiment does not isolate the revision's causal effect on accuracy. The histories overlap but are not literal nested prefixes despite that wording in the original protocol.

The large study uses all 20,000 UCI Letter observations with 16 features and 26 classes [17]. A stratified split with seed 20261015 gives 15,000 training and 5,000 test observations, and train-only MinMax scaling and chi-squared selection retain 12 features. The fixed 16 forests combine 64/128/192/256 trees with two feature-sampling settings and two model seeds. They are genuinely trained models and no padding or duplicate payload is introduced to reach a storage target.

## 4.2 Comparators and budget fairness

The exact comparators solve the same complete preservation obligation. The robust greedy policy adds the best grounded coverage gain per byte, and the stronger local policy also reverse-deletes from a full store and tries exchanges that add one artifact followed by pruning. These are our implementations and the local policy allows at most two exchange passes, so a gap against it is not a claim against every local-search algorithm. Target snapshots keep protected outputs directly. A manual frontier control chooses the smallest feasible workflow-specific frontier, which tests whether expert knowledge can match automatic selection.

The HYPPO rank and frequency methods use a pinned released component and HELIX uses a documented rule adapter. They share the executor and byte accounting with the retention methods but preserve their reuse-oriented selection criteria. In the new cloud grid these adapters receive the exact minimum total byte budget, whereas robust heuristics and snapshots receive the full candidate allowance when comparing complete store sizes. The scientific matrix instead gives all methods the same fixed budget at each point. Comparisons must therefore report their budget and objective rather than collapse every method into one universal ranking.

The native DVC study uses DVC 3.67.1 and gives it the certified optimal retained payloads without charging selection time. One downstream batch stage avoids a separate Python startup for every model, and correctly retained target cache objects can let DVC skip that stage. Before each trial both methods have the same paid physical payloads and no unpaid downstream cache objects, with distinct-inode byte checks and hard links used for common retained files. DVC configuration is recorded separately from the common payload objective. Both authored workers can use the exact file-export helper.

The later encoded storage study retains the previous flat optimum as a candidate, adds exact prefix and suffix deltas with their bases and headers charged, and measures a joint archive of retained canonical payloads. The final selector computes the flat exact, delta exact, flat local, delta local and separately compressed choices and keeps the smallest complete store. We compare it with the previous flat and delta implementation, a joint archive of the flat optimum, and local search with all offered formats. The first

three-candidate version lost three conditions to the strengthened local control; adding both local plans was a disclosed correction after that result. Every row in the later table uses the final five-candidate selector, and the original losses remain saved in the artifact [4]. The exact reference only certifies the declared flat or delta inventories, not a global optimum over every jointly compressed retained set.

The later storage catalogue has 216 primary, 48 scientific transfer, 72 three-seed cloud and 108 retinal conditions, plus two Letter conditions whose complete paired storage values are unavailable. These conditions share captures and were chosen for this project, so win fractions describe the catalogue rather than an estimated production probability. All stored byte totals include metadata, mandatory roots and delta bases. Saved file and export checks confirm exact identity but are not new full-grid model replay requests. The joint archive is decoded into memory and uses a temporary retained store for replay; its compression time and peak memory were not benchmarked.

The broader native DVC census has 72 captures from six datasets, four recipes and three history lengths. Three loss contracts, two request sizes and command line or warm API access make 864 configurations, each timed in three paired randomized-order repetitions. The methods receive the same certified optimal retained payloads. The elapsed call includes requested-file inventory and hash verification but excludes capture, planning and setup, and OS caches are not flushed. A separately coded direct executor supplies a control for the cost of DVC orchestration [4].

### 4.3 Machines, repetitions and recorded outcomes

The shared loss study runs one fixed seed block on each of three Linux GitHub runners exposing four virtual CPUs on AMD EPYC 7763 hosts. The native study uses runners exposing four CPUs with EPYC 9V74 for one block and EPYC 7763 for two. Python is 3.12.15 with NumPy 2.3.5 and SciPy 1.17.0 and scikit-learn 1.8.0, and numerical execution is limited to one BLAS thread. These are separate runner blocks of the same constructed workload design rather than multi node scale-out.

Each cloud planner has three timed calls per case and the earlier 48-condition replanning study has five. General solver calls use a 30 s limit and a requested relative MIP gap of zero, with 120 s allowed in Letter. The polynomial flow route does not enforce a hard interruption deadline. A native recovery comparison has three paired trials with deterministically randomized method order. Its primary elapsed time includes cold subprocess and CLI startup and final common SHA-256 verification, while setup and planning are excluded and worker timings and CPU are logged separately. OS page caches are not flushed. The final native revision follows earlier exploratory corrections and is not pooled with them.

Every actual request records whether its expected target identities were recovered, whether a deficient store was correctly blocked and whether an unexpected outcome occurred. When methods select identical stores they share explicitly attributed execution evidence, so method totals must not be summed as new physical requests. Ratios are computed within a history from method medians and then summarized across histories. These descriptive ratios and repeated requests do not establish population significance.

The older concurrency control uses a Linux Xeon Platinum 8573C host with a CPU quota of eight cores and Python 3.12.14. It randomizes 54 trials across three policies and two data sizes and three worker counts and three repeats. All five versions are warmed before a timed trial and each request has a fresh restorer and memo. Service latency includes environment validation and disk recovery, while response latency also includes queue wait. Throughput excludes capture and planning, and its resource sampler records CPU and RSS for the whole benchmark process. This is not a concurrency evaluation of the newer native DVC integration.

**Table 2: Storage comparison against the tested strong local policy. Each row has 36 cases. The manual best frontier ties every minimum in both rows and the median full grid byte reduction is zero.**

| Loss grouping | Wins | Ties | Losses | Max. reduction |
|---|---|---|---|---|
| Separate | 6 | 30 | 0 | 0.1942% |
| Shared | 9 | 27 | 0 | 48.7380% |

## 5 Results

### 5.1 Storage choices under shared preprocessing loss

The two exact formulations agree on all 72 paired history and contract minima, and the recorded repeated planning calls contain 216 checked flow proof records. Under separate entry points RecoverML uses fewer bytes than local search in six cases and ties in 30, but each difference is only 129 serialized bytes. Under shared loss it wins in nine and ties in 27, with no storage losses in this grid. Table 2 keeps the complete case counts visible and Figure 2 shows every shared-loss case.

The wins occur in fused Wine histories with 16 or 64 versions and fused Breast Cancer histories with 64 versions across all three seeds. In the largest relative case RecoverML retains 138,618 bytes and local search retains 270,411, a saving of 131,793 bytes or 48.7380%. This is a meaningful decision gap for that topology but is about 132 KB in absolute size rather than the separate multi-GB result.

Both standardized views can be required together to replace many models and predictions, so a single-artifact exchange can miss the better joint frontier. Recording a concatenation supplies that joint value as one candidate and removes the gap in all 18 controls. The manual best frontier also matches every exact minimum. The evidence therefore identifies interacting retention choices and a capture-boundary effect rather than a universal need for an exact solver.

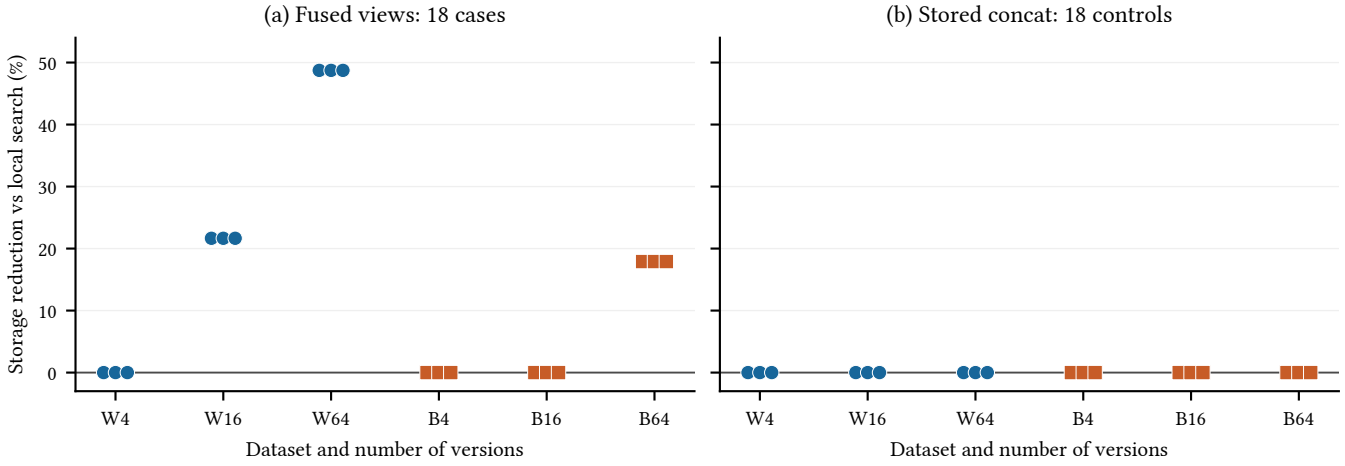



**Figure 2: All 36 shared-loss storage comparisons. W and B denote Wine and Breast Cancer and the number is the version count. Each setting has three seed points. The fused boundary exposes nine local-search gaps, while every explicit-concat control ties. Reduction is measured relative to local retained bytes, including common fixed accounting.**

## 5.2 Coverage and the meaning of a complete contract

At the cloud grid's exact minimum budget RecoverML is complete in 36/36 histories per loss grouping, compared with 18/36 for HYPPO rank and 21/36 for frequency and 15/36 for the HELIX rule. The robust full store controls remain feasible but can retain more bytes. Across physically unique cloud requests there are 16,110 exact results and 3,542 expected blocks out of 19,652, with zero unexpected outcomes. RecoverML's attributed requests are all exact, and the blocks belong to deficient adapter stores.

The separate scientific matrix shows why complete coverage and the number of partially recovered requests should not be confused. Table 3 reports both quantities across its fixed budgets. RecoverML and local search are complete in 128 of 240 conditions each, but local search returns more individual targets across the whole grid. The exact complete-preservation policy does not optimize partial coverage below its feasibility threshold. At full budget strong local search matches all 48 exact storage minima, so this older study does not establish a storage-choice advantage over that baseline.

**Table 3: Scientific fixed budget matrix. Every method has 240 conditions and 4,560 requests. An expected block can be correct under the contract, and individual request coverage is a different objective from complete preservation.**

| Method | Complete conditions | Exact requests |
|---|---|---|
| RecoverML exact | 128 | 1,734 |
| Scenario exact | 128 | 1,734 |
| Robust greedy | 112 | 2,221 |
| Robust local | 128 | 2,324 |
| HYPPO rank | 104 | 1,989 |
| HYPPO frequency | 100 | 1,856 |
| HELIX rule | 96 | 1,700 |
| Target snapshots | 112 | 1,862 |

## 5.3 Planning cost and component checks

In the new cloud grid the median paired local-to-RecoverML planning ratio is 11.92 under separate loss and 16.88 under shared loss, and the scenario-to-RecoverML ratios are 2.28 and 1.73. These compare planning wall times rather than artifact recovery. The older 48-condition timing study retains the original minima and gives a 4.20 median ratio against local search, with RecoverML faster in 44 conditions and slower in four.

Figure 3 separates the 36 DAG-eligible cases from 12 general fallback cases. The eligible subset is faster than local search in every case with a median ratio of 6.27, while the fallback median is 1.64 against local and 0.93 against scenario exact. A ratio below one means RecoverML is slower. The guard gives a useful specialization but is not a fastest-solver oracle for other structures.

A separate finite guard validation enumerates 2,275 DAG specifications with four artifacts and checks 18,200 closure comparisons for fixed stores and 127,400 minimum cut optima against exhaustive search, with no disagreement. These finite checks support the stated model and do not establish correctness for arbitrary unrecorded workflows.

The component study has 154 recorded rows including 110 rows under the same contract, and all 99 certified same contract objective comparisons agree. All 1,452 enumerated fault certificate checks and six indexed versus reference chain comparisons also agree. Decomposition is slower than the undecomposed route in six of eight settings from public recipes, so a correct split does not guarantee lower overhead. Changing logical identity is a change of contract rather than an optimization ablation for the same objective. The separate annotation study with 5,760 conditions changes selected sets in 298/4,608 cost profile comparisons and 312/2,880 access weight comparisons, which shows that the

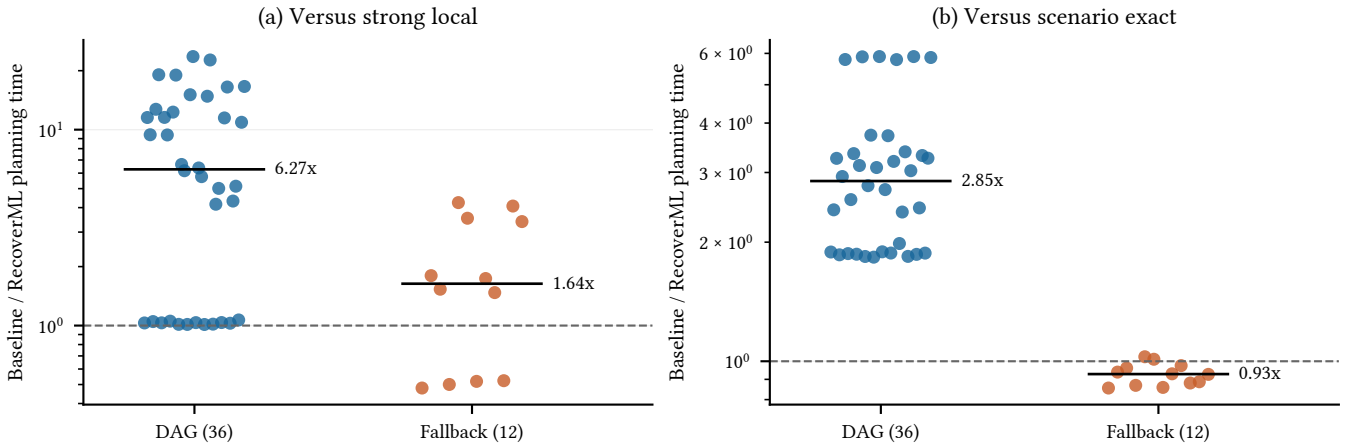



**Figure 3: All 48 earlier-condition planning ratios, with one point per condition and a black median bar. Each method has five planning repetitions per condition. Values above one favor RecoverML, and the fallback's slower median against scenario exact and all four slower local comparisons remain visible.**

adapter outcomes depend on their declared annotations. These are structural and planner checks and not additional timed disk recoveries.

## 5.4 Native recovery at identical retained storage

The final native comparison completes 216 full history subprocess commands with 12,096 exact target hash checks. RecoverML has lower median elapsed time in all 36 histories, and Figure 4 separates the two recovery strata rather than pooling different kinds of work into one headline speedup.

For the 18 histories needing downstream computation the median paired DVC-to-RecoverML ratio is 2.1856, with ratios from 1.7215 to 2.5109. For the 18 histories whose targets are already retained it is 9.9067, with ratios from 8.9527 to 11.3402. The medians of the per history absolute times are 1.663 s for RecoverML and 4.146 s for DVC in the recomputation stratum, and 0.0633 s versus 0.6056 s in the retained target stratum. A ratio of these aggregate medians need not equal the median of paired ratios.

The direct export path avoids numerical startup when replay is unnecessary and both workers have access to the same helper. DVC still pays the orchestration cost of its native command and stage handling, so the large retained target ratio should be read as a property of these cold command integrations. An earlier revision with the corrected paid cache was mixed: RecoverML won all 18 recomputation cases but DVC won all 18 retained target cases. The export improvement was added after that result and evaluated in a fresh full grid. Earlier versions remain separate evidence and are not pooled with the final revision.

## 5.5 Complete encoded storage comparison

Table 4 reports the later five-candidate choice against four storage controls. The total denominator stays at 446, although one Letter condition has only a selected store and another lacks the original archive, so neither supports a complete measured pair. The choice never increases bytes in the 444 measurable comparisons because it includes the older plan and both local plans as fallback candidates. The older three-candidate joint choice did lose three cases to the strengthened equal-format local control: it had 20 wins, 421 ties and three losses. Those losses prompted the two added local candidates, and the correction is disclosed rather than counted as a new independent confirmation [4].

Against the previous flat optimum the largest reduction is 89.84%, from 1,457,715 to 148,050 bytes in `digits/pca_neighbors/v16/broad`. The largest gap against the equal-format local control is 70.17%, from 118,126 to 35,242 bytes in `shared/wine-s20261016-v64-concat0`. Most of that local gap already existed before joint compression, so it is not evidence of a new 70% compression gain. The median saving against local search is zero and only nine of 446 conditions reach 10% savings against that control. The inherited broad improvement test, which required at least 10% storage saving in 90% of histories per stratum, passes in zero of 72 strata.

The equal-format exact reference ties the measured flat and delta optima under their declared candidate inventories, and the manual best frontier ties all 36 minima in the earlier shared-loss study. An exact minimum cannot be strictly beaten under identical rules on the same byte objective. The joint archive changes those rules, while a global minimum over every possible jointly compressed retained set has not been established. The result is a systems gain in particular captured conditions, with no measured storage regression for the corrected portfolio, rather than strict dominance over every close comparator.

The independent release audit rebuilt the archive controls from saved canonical payloads and reproduced both local selectors. It reconciled 5,061 selected-file checks and 3,531 retained-target export checks, including repeated artifacts, and all 29 cloud jobs and the final audit passed. These checks are evidence for the saved stores and file identities, not 8,592 new full-model replay requests. No full-catalogue compression time, random access latency or peak memory advantage is claimed [4].

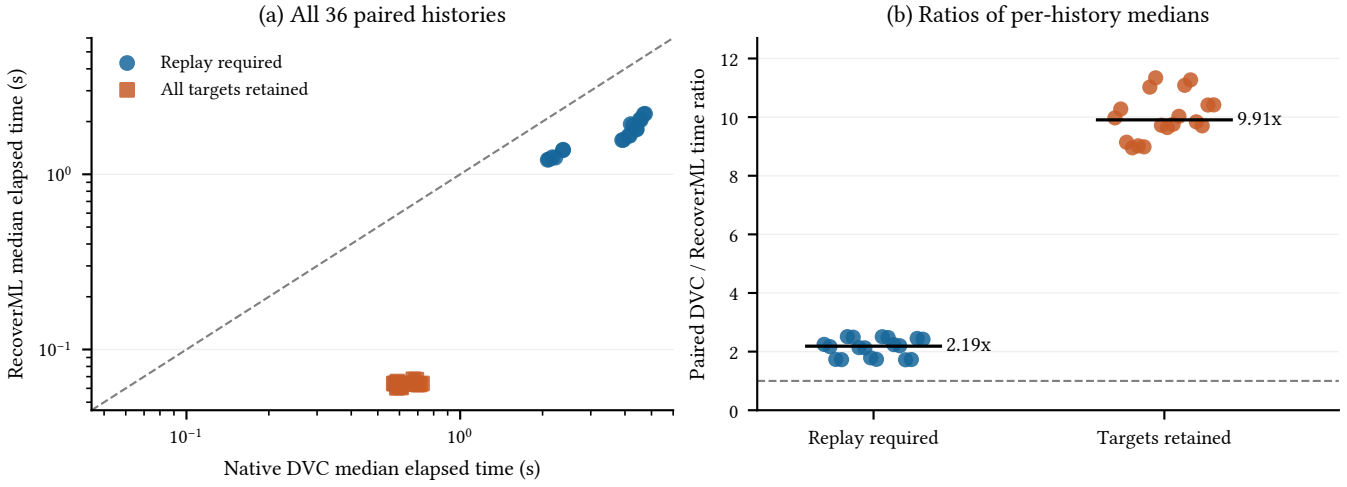



**Figure 4: Final native DVC comparison at identical optimal paid payloads. Panel (a) plots each history's median time from three paired trials and the dashed line is equality. Panel (b) shows all per history ratios with their stratum medians. Startup and verification are included and setup and planning are excluded; OS caches are not flushed.**

**Table 4: Complete storage outcomes for the final five-candidate choice. Smaller, equal and larger compare paid bytes under the listed control. The two Letter cases remain unmeasured in every row. The 10% column counts conditions with at least a 10% reduction out of all 446 conditions.**

| Comparator | Smaller | Equal | Larger | Unmeasured | Median saving | 10%+ |
|---|---|---|---|---|---|---|
| Previous flat optimum | 244 | 200 | 0 | 2 | 0.094% | 52 |
| Previous flat or delta choice | 214 | 230 | 0 | 2 | 0% | 47 |
| Joint archive of flat optimum | 233 | 211 | 0 | 2 | 0.056% | 5 |
| Local search with all offered formats | 20 | 424 | 0 | 2 | 0% | 9 |

## 5.6 Broader native DVC recovery census

In the later census RecoverML exceeds the prespecified 1.10 times elapsed-time threshold against the tested native DVC integration in all 864 configurations. The median paired DVC-to-RecoverML ratio is 1.6952 for the 432 command line configurations and 48.3212 for the 432 warm API configurations. All 7,776 trials restore requested files exactly and the ledger checks 54,540 requested-file hashes. Latest and all-version requests coincide when a history has one version, so these counts do not represent independent workloads [4].

The direct executor control narrows the interpretation. Its median elapsed time divided by RecoverML time is 0.9929, slightly favoring the direct control, and RecoverML reaches a 1.10 times improvement in only 21 of 864 direct comparisons while being at least 1.10 times slower in 48. The native DVC difference therefore mainly measures orchestration in these integrations. Reusing DVC repository objects, other cache policies and remote recovery were not tested. This census supports a consistent advantage against the specified native path across the authored catalogue, but it does not establish faster model training or a production win probability.

## 5.7 Historical processing change and large payloads

The retinal study completes 10,920 exact requests across 540 method rows with zero unexpected errors, and Figure 5 shows all 108 storage comparisons. Exact retention saves bytes against target snapshots in 90 cases and ties in 18. It improves on simple greedy in 12 and ties in 96, but strong local search ties all 108 optima. A simple frontier rule matches those minima as well, so the application supports the preservation use case without demonstrating a need for the general optimizer.

The largest retinal snapshot saving is 12,316,663 bytes in an intact forest history whose targets include investigation features. In a separate native control the legacy module is physically absent before and after reproduction and all 32 protected targets are recovered exactly. A manually retained DVC frontier matches the common 2,523,881-byte objective. The main grid's declared restrictions and this physical removal control should not be described as the same failure experiment.

For Letter, preprocessing loss reduces the canonical store from 3,669,545,821 to 4,829,269 bytes, saving 3,664,716,552 bytes or 99.8684%. With gzip the corresponding sizes are 291,175,536 and 568,375 bytes, saving 290,607,161. Figure 6 gives both representations and the broader-loss control. Strong robust heuristics match

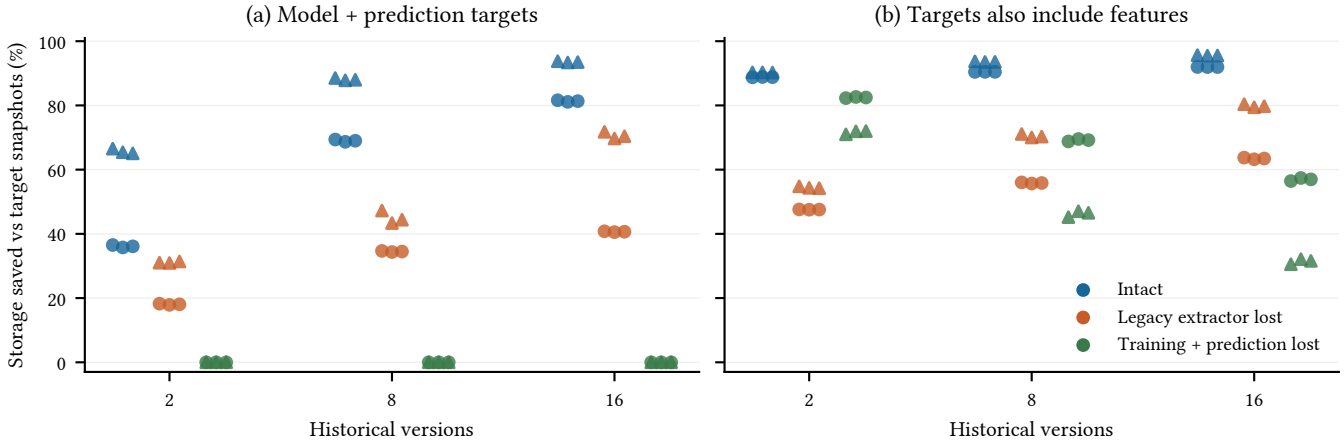



**Figure 5: All 108 retinal storage conditions. Color gives the availability contract and circles and triangles denote logistic regression and forests. Small offsets show the three seeds. Adding investigation features changes the preservation obligation, while broad training and prediction loss can require target snapshots. Repeated conditions share the same 20 public images.**

the preprocessing optimum and broad loss yields zero canonical or gzip saving. Native-pickle and native-gzip controls are recorded separately because their byte identities differ from the canonical representation.

The actual Letter replay ledger contains 48 exact requests across three unique stores under all preprocessing groups disabled, a stronger stress than its $k = 1$ selection contract. The broad-loss entries are structural checks and are not additional actual runtime requests. In the descriptive preprocessing replay, mean frontier recovery is 6.65 s per version compared with 2.12 s for snapshot loading. The evaluation includes all 16 trained forests and their held-out accuracy ranges from 95.0% to 95.64%, but improving predictive accuracy is not the retention objective.

## 5.8 Earlier concurrency and latency control

Figure 7 regenerates throughput and latency curves from the earlier prototype's saved records. All 5,400 requests are exact across 54 trials, and the audit also reconciles 2,129 resource samples and 39,600 operation records. The earlier RecoverML V2 policy and target snapshots select the same retained files on these settings, so their small timing differences do not demonstrate different retention quality.

At 10,000 rows the V2 policy's median throughput falls from 817.49 exact requests/s with one worker to 412.20 with four, while median per-trial service P99 rises from 1.96 ms to 24.53 ms. Full replay instead rises from 32.45 to 40.06 requests/s. The retained file workload therefore does not scale monotonically with more threads in this host and cache setting, which is consistent with overhead and contention but does not isolate their causes. Response time also includes queued work and must not be read from the service CDF. Three short trials provide descriptive tail samples rather than stable production P99 guarantees.

# 6 Discussion and Reproducibility

The contract prioritizes complete recovery and minimum retained bytes, while recovery latency remains a separate cost. Manual frontier ties and faster snapshot loads show why no policy wins every objective.

The histories use public data but their processing graphs and losses are constructed, and the shared-loss pilot and earlier native revisions informed later experiments. The encoded-storage catalogue also reuses captures and its final candidate choice was corrected after a stronger local control revealed three losses. Seeds, versions, contracts and requests share data, so the paired summaries are descriptive rather than population estimates. The retinal adaptation is not the private EyeAI workflow and Letter's byte gain depends on representation. Annotation effort, deployment cost and the prevalence of retired capabilities remain unmeasured.

The artifact preserves source hashes and captured identities and raw timestamps and solver states with the analysis scripts [4]. Fresh capture and replay on three runners do not establish portable replay across arbitrary software changes. Six original nonforest scientific captures lack a closed global end of run source guard after an interruption, although their snapshots and request guards and raw reconciliation remain. One original Letter archive container is missing; four recovered frontier payloads match their original hashes in a separately identified replacement archive. Available-file integrity and original-container completeness are separate checks.

The supplied script regenerates the original figures from pinned recorded inputs, while the later storage and recovery ledgers are linked in the repository artifact [4]. New benchmark executions require a fresh output directory. No GPU was used in these studies. The joint archive decodes retained content into memory, and its latency, compression cost and peak-memory tradeoff remain unmeasured. Larger neural training states, distributed scale out and crash consistency require separate implementation and evaluation.

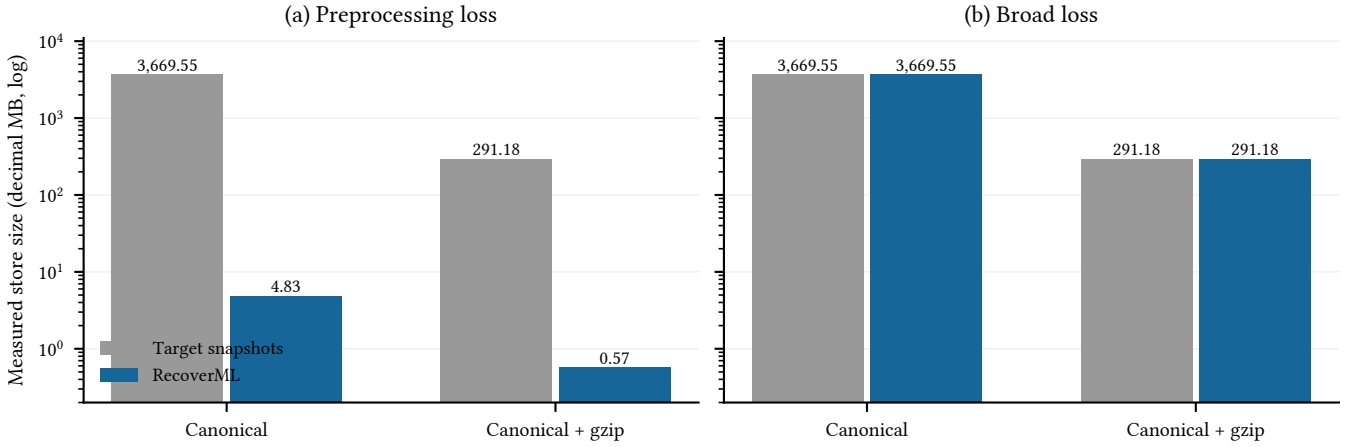



**Figure 6: Measured Letter store sizes in decimal MB. Each representation is compared with its own target-snapshot baseline and values are shown on a logarithmic axis. The uncompressed saving exceeds 3.66 GB, the measured gzip saving is 290.61 MB, and broad loss removes both savings.**

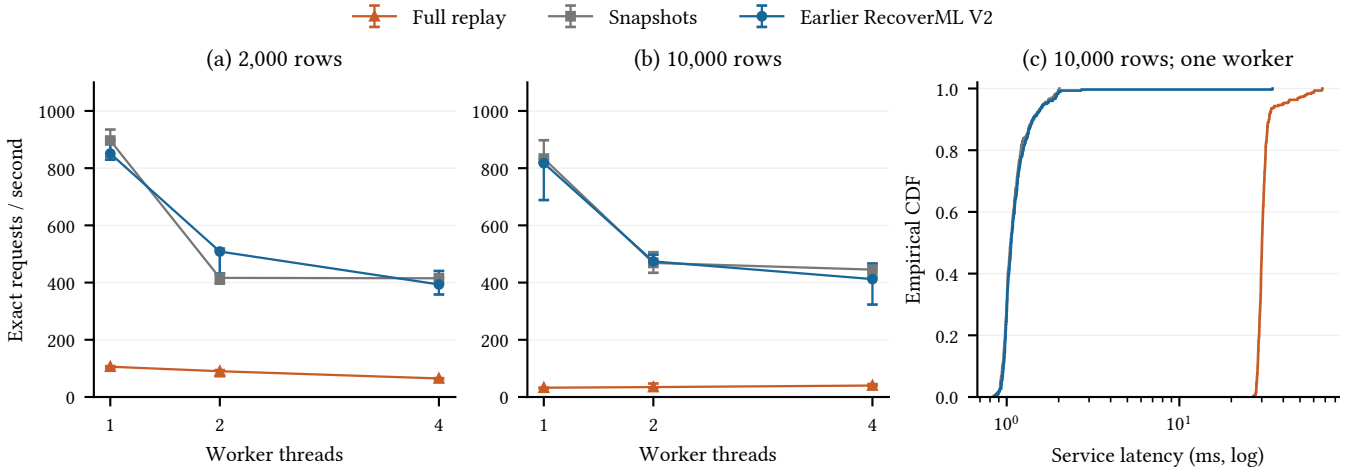



**Figure 7: Earlier prototype concurrency control, not the final native DVC study. Throughput points are medians of three trials and whiskers show their observed minima and maxima. The CDF pools 300 exact one worker requests per method at 10,000 rows and describes service latency, which excludes queue wait. Page caches are warm and each request has a fresh memo.**

## 7 Conclusion

RecoverML selects files under an explicit historical preservation contract and checks their recovery and byte identities. General exact formulations and a guarded DAG specialization support different recorded graph shapes. Shared preprocessing loss exposes storage choices missed by the tested local heuristic, while a stored shared intermediate removes that gap and manual frontiers still match the optimum. The later flat, delta and joint archive choice reduces storage against the previous flat optimum in 244 of 444 measurable conditions and against equal-format local search in 20, with 424 local ties and no measured losses after the disclosed correction. The broader native DVC census finds a consistent elapsed-time advantage at equal retained payloads, while a direct executor is near parity. These results establish selective storage and orchestration benefits within the tested contracts, and independently collected workflows remain needed to judge how often those benefits arise in practice.

## References

[1] Behrouz Derakhshan, Alireza Rezaei Mahdiraji, Ziawasch Abedjan, Tilmann Rabl, and Volker Markl. 2020. Optimizing Machine Learning Workloads in Collaborative Environments. In *Proceedings of the ACM SIGMOD International Conference on Management of Data*. Association for Computing Machinery, New York, NY, USA, 1701–1716. doi:10.1145/3318464.3389715

[2] Behrouz Derakhshan, Alireza Rezaei Mahdiraji, Zoi Kaoudi, Tilmann Rabl, and Volker Markl. 2022. Materialization and Reuse Optimizations for Production Data Science Pipelines. In *Proceedings of the ACM SIGMOD International Conference on Management of Data*. Association for Computing Machinery, New York, NY, USA, 1962–1976. doi:10.1145/3514221.3526186

[3] DVC contributors. 2026. *Data Version Control: Reproduction and Frozen Stages*. https://doc.dvc.org/command-reference/repro See also the freeze command reference. Accessed 17 October 2026; experiments use DVC 3.67.1.

[4] Abdul Hannaan. 2026. RecoverML Reproducibility Artifact. https://github.com/Hannaancode/RecoverML/tree/efae825be13ffe487b915a4580cdc16b4c5e95c6 Source and recorded experimental evidence at commit efae825, accessed 28 October 2026.

[5] Adam Hoover, Valentina Kouznetsova, and Michael Goldbaum. 2000. Locating Blood Vessels in Retinal Images by Piecewise Threshold Probing of a Matched Filter Response. *IEEE Transactions on Medical Imaging* 19, 3 (2000), 203–210. doi:10.1109/42.845178

[6] Paras Jain, Ajay Jain, Aniruddha Nrusimha, Amir Gholami, Pieter Abbeel, Kurt Keutzer, Ion Stoica, and Joseph E. Gonzalez. 2020. Checkmate: Breaking the Memory Wall with Optimal Tensor Rematerialization. *Proceedings of Machine Learning and Systems* 2 (2020), 497–511. https://proceedings.mlsys.org/paper_files/paper/2020/file/0b816ae8f06f8dd3543dc3d9ef196cab-Paper.pdf

[7] Antonios Kontaxakis, Dimitris Sacharidis, Alkis Simitsis, Alberto Abelló, and Sergi Nadal. 2024. HYPPO: Using Equivalences to Optimize Pipelines in Exploratory Machine Learning. In *Proceedings of the IEEE International Conference on Data Engineering*. IEEE, 221–234. doi:10.1109/ICDE60146.2024.00024

[8] Antonios Kontaxakis, Dimitris Sacharidis, Alkis Simitsis, Alberto Abelló, and Sergi Nadal. 2025. Hyppo: Efficient Discovery and Execution of Data Science Pipelines in Collaborative Environments. In *Proceedings of the International Conference on Extending Database Technology*. OpenProceedings.org, 1134–1137. doi:10.48786/edbt.2025.107

[9] Zhaoheng Li, Supawit Chockchowwat, Ribhav Sahu, Areet Sheth, and Yongjoo Park. 2024. Kishu: Time-Traveling for Computational Notebooks. *Proceedings of the VLDB Endowment* 18, 4 (2024), 970–985. doi:10.14778/3717755.3717759

[10] Zhiwei Li, Carl Kesselman, Tran Huy Nguyen, Benjamin Yixing Xu, Kyle Bolo, and Kimberley Yu. 2025. From Data to Decision: Data-Centric Infrastructure for Reproducible ML in Collaborative eScience. arXiv:2506.16051. doi:10.48550/arXiv.2506.16051

[11] Naga Nithin Manne, Shilvi Satpati, Tanu Malik, Amitabha Bagchi, Ashish Gehani, and Amitabh Chaudhary. 2022. CHEX: Multiversion Replay with Ordered Checkpoints. *Proceedings of the VLDB Endowment* 15, 6 (2022), 1297–1310. doi:10.14778/3514061.3514075

[12] MIT 6.854. 2016. *Lecture Notes: Advanced Flow Algorithms*. https://courses.csail.mit.edu/6.854/16/Notes/n9-advanced_flow_algorithms.html Accessed 17 October 2026.

[13] Paul Saikko, Johannes P. Wallner, and Matti Järvisalo. 2016. Implicit Hitting Set Algorithms for Reasoning Beyond NP. https://www.dbai.tuwien.ac.at/user/wallner/files/SaikkoWJ2016.pdf Author version with full proofs and extended related work.

[14] Mariko Sakashita, Kazuhisa Makino, and Satoru Fujishige. 2007. *Minimum Cost Source Location Problems with Flow Requirements*. Technical Report METR 2007–39. University of Tokyo. https://www.keisu.t.u-tokyo.ac.jp/data/2007/METR07-39.pdf

[15] scikit-learn developers. 2026. *Model Persistence*. https://scikit-learn.org/1.8/model_persistence.html scikit-learn 1.8 documentation. Accessed 17 October 2026.

[16] scikit-learn developers. 2026. *Toy Datasets*. https://scikit-learn.org/1.8/datasets/toy_dataset.html scikit-learn 1.8 documentation. Accessed 17 October 2026.

[17] David Slate. 1991. Letter Recognition. UCI Machine Learning Repository. doi:10.24432/C5ZP40 Dataset.

[18] Doris Xin, Stephen Macke, Litian Ma, Jialin Liu, Shuchen Song, and Aditya Parameswaran. 2018. HELIX: Holistic Optimization for Accelerating Iterative Machine Learning. *Proceedings of the VLDB Endowment* 12, 4 (2018), 446–460. doi:10.14778/3297753.3297763

[19] Siyuan Zhuang, Stephanie Wang, Eric Liang, Yi Cheng, and Ion Stoica. 2023. ExoFlow: A Universal Workflow System for Exactly-Once DAGs. In *Proceedings of the USENIX Symposium on Operating Systems Design and Implementation*. USENIX Association, Boston, MA, USA, 269–286. https://www.usenix.org/conference/osdi23/presentation/zhuang